TravelMemory

MERN Stack Deployment on AWS

Terraform + Ansible | Implementation README

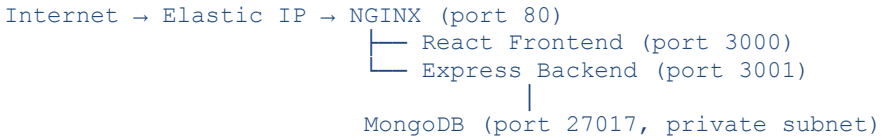
Author: Avinash Sain | GitHub: github.com/Avinashsain

1. Overview

This document covers the end-to-end deployment of the TravelMemory MERN application on AWS. Infrastructure is provisioned with Terraform and the application is configured and deployed using Ansible.

Application	TravelMemory (MERN Stack)
Infrastructure	Terraform >= 1.3.0
Configuration	Ansible >= 2.14
Cloud Provider	AWS (us-east-1)
Web Server AMI	Ubuntu 24 (ami-0b6d9d3d33ba97d99)
Database	MongoDB 7.0 (private EC2)
State Backend	S3 + DynamoDB Lock

2. Architecture



VPC	10.0.0.0/16
Public Subnet	10.0.1.0/24 – Web Server + NAT Gateway
Private Subnet	10.0.2.0/24 – MongoDB Server (no public IP)
Internet Gateway	Public subnet outbound/inbound
NAT Gateway	Private subnet outbound only
Web EC2	t3.small, Ubuntu, Elastic IP
DB EC2	t3.small, Ubuntu, private IP only

3. Prerequisites

- AWS CLI installed and configured (aws configure)
- Terraform >= 1.3.0 installed
- Ansible >= 2.14 installed
- SSH key pair at ~/.ssh/travelmemory

```
# Generate SSH key pair
ssh-keygen -t ed25519 -f ~/.ssh/travelmemory -C 'travelmemory'

# Get your public IP for trusted_ip in tfvars
curl ifconfig.me
```

4. Project Structure

```
TravelMemory/
├── terraform/
│   ├── main.tf           # VPC, subnets, IGW, NAT Gateway, route tables
│   ├── ec2.tf            # EC2 instances, key pair, Elastic IP
│   ├── security.tf       # Security groups, IAM role
│   ├── backend.tf        # S3 backend, DynamoDB lock table
│   ├── variables.tf      # Input variable declarations
│   ├── outputs.tf        # Output values
│   └── terraform.tfvars  # Your values (git-ignored)
└── ansible/
    ├── ansible.cfg       # Ansible settings
    ├── inventory.ini     # Host definitions
    ├── site.yml          # Master playbook
    ├── group_vars/
    │   └── all.yml       # Vault-encrypted secrets
    └── roles/
        ├── webserver/    # Node.js, NGINX, React, PM2
        └── database/     # MongoDB 7.0 + auth hardening
```

5. Part 1 – Infrastructure Setup with Terraform

5.1 AWS CLI Configuration

```
aws configure
# AWS Access Key ID: <your key>
# AWS Secret Access Key: <your secret>
# Default region: us-east-1
# Default output format: json
```

5.2 Terraform Initialization

```
cd terraform/
cp terraform.tfvars.example terraform.tfvars
# Edit terraform.tfvars – set trusted_ip, public_key_path

terraform init
```

5.3 S3 Backend + DynamoDB State Locking

Terraform state is stored remotely in S3 with DynamoDB locking to prevent concurrent applies.

```
# Step 1 – Create DynamoDB lock table
terraform apply -target=aws_dynamodb_table.terraform_lock

# Step 2 – Migrate local state to S3
terraform init -migrate-state
```

S3 Bucket	streamingappbucketb15
State Key	travelmemory/terraform.tfstate
DynamoDB Table	travelmemory-terraform-lock
Encryption	AES256 server-side
Lock Attribute	LockID

5.4 Plan and Apply

```
terraform plan
terraform apply
```

5.5 Terraform Outputs

```
terraform output

web_server_public_ip = "100.49.217.31"
db_server_private_ip = "10.0.2.116"
vpc_id               = "vpc-xxxxxxxxx"
```

5.6 AWS Resources Created

Resource	Details
VPC	10.0.0.0/16, DNS enabled
Public Subnet	10.0.1.0/24, us-east-1a

Private Subnet	10.0.2.0/24, us-east-1a
Internet Gateway	Attached to VPC
NAT Gateway	In public subnet, EIP attached
Web Server EC2	t3.small, Ubuntu, public subnet
DB Server EC2	t3.small, Ubuntu, private subnet
Elastic IP	Static IP for web server
Web Security Group	Ports 22, 80, 443, 3000, 3001
DB Security Group	Port 27017 from web SG only
IAM Role	SSM + CloudWatch permissions
S3 State Backend	Encrypted, versioned
DynamoDB Lock	Prevents concurrent applies

6. Part 2 – Configuration and Deployment with Ansible

6.1 Update Inventory

After terraform apply, update ansible/inventory.ini with the new IPs:

```
[webservers]
web ansible_host=100.49.217.31 ansible_user=ubuntu
ansible_ssh_private_key_file=~/.ssh/travelmemory

[databases]
db ansible_host=10.0.2.116 ansible_user=ubuntu \
   ansible_ssh_private_key_file=~/.ssh/travelmemory \
   ansible_ssh_common_args='-o ProxyJump=ubuntu@100.49.217.31'

[all:vars]
ansible_ssh_common_args='-o StrictHostKeyChecking=no'
ansible_python_interpreter=/usr/bin/python3
```

6.2 Ansible Vault – Encrypt Secrets

```
# Create vault password file
echo 'your_vault_password' > ~/.vault_pass
chmod 600 ~/.vault_pass

# Encrypt each secret
```

```
ansible-vault encrypt_string 'tmuser' --name 'mongo_app_user' --vault-password-file
~/.vault_pass --encrypt-vault-id default
ansible-vault encrypt_string 'StrongPassword123' --name 'mongo_app_password'
--vault-password-file ~/.vault_pass --encrypt-vault-id default
ansible-vault encrypt_string 'AdminPassword123' --name 'mongo_admin_password'
--vault-password-file ~/.vault_pass --encrypt-vault-id default
```

Paste encrypted values into group_vars/all.yml:

```
mongo_app_user: !vault |
          $ANSIBLE_VAULT;1.1;AES256
          <encrypted value>
```

6.3 Test Connectivity

```
cd ansible/
ansible all -m ping

# Expected output:
web | SUCCESS => { "ping": "pong" }
db  | SUCCESS => { "ping": "pong" }
```

6.4 Run the Playbook

```
ansible-playbook site.yml
```

6.5 What Each Role Does

Webserver Role

- apt update and install Node.js, NPM, NGINX, Git
- Clone TravelMemory repo from GitHub (master branch)
- Deploy backend .env with MONGO_URI
- npm install backend dependencies
- Deploy frontend .env with REACT_APP_BACKEND_URL
- npm install and npm run build React frontend
- Install PM2 and serve globally
- Start backend (port 3001) and frontend (port 3000) with PM2
- Configure NGINX reverse proxy on port 80
- Harden SSH – disable root login and password auth

Database Role

- apt update and install gnupg, curl, python3-pymongo
- Add MongoDB 7.0 GPG key and apt repository
- Install mongodb-org packages
- Configure bindIp to listen on private IP
- Start MongoDB without auth
- Create admin user and application user (tmuser)

- Enable MongoDB authentication
- Verify authentication works
- Harden SSH

7. Application Access

Service	URL
Frontend (via NGINX)	http://100.49.217.31
Frontend (direct)	http://100.49.217.31:3000
Backend API	http://100.49.217.31:3001
Trips endpoint	http://100.49.217.31:3001/trip

```
# Test backend API
curl http://100.49.217.31:3001/trip

# Test frontend
curl http://100.49.217.31
```

8. Security Hardening

Measure	Implementation
MongoDB private subnet	No public IP, unreachable from internet
DB SSH access	Only via web server ProxyJump
SSH key-pair only	PasswordAuthentication no on both servers
Root login disabled	PermitRootLogin no on both servers
MongoDB auth	Role-based users, authorization enabled
Secrets encrypted	Ansible Vault for all passwords
State encrypted	S3 AES256 server-side encryption
State locked	DynamoDB LockID prevents concurrent applies
IAM least privilege	SSM + CloudWatch only, no admin access

Security groups	Ports locked to specific sources

9. Teardown

To destroy all AWS resources and stop incurring charges:

```
cd terraform/
terraform destroy
```

This will remove: EC2 instances, VPC, subnets, IGW, NAT Gateway, Elastic IPs, security groups, IAM roles, and DynamoDB lock table. The S3 bucket has prevent_destroy = true and must be deleted manually.

10. Author

Avinash Sain
GitHub: https://github.com/Avinashsain
Repository: https://github.com/Avinashsain/TravelMemory26